School of Technology and Engineering (SOTE)

MAL2020

Computing Group Project

Final Report

Team Member	Student ID	Programme / Major	Email
Eu Jin Yang	BSSE2506021	BSc (Hons) Computer Science (Software Engineering)	BSSE2506021@peninsulamalaysia.edu.my
Ng Kean Chun	BSSE2506036		BSSE2506036@peninsulamalaysia.edu.my
Teoh Yi Shan	BSSE2506022		BSSE2506022@peninsulamalaysia.edu.my
Yong Ken	BSSE2506025		BSSE2506025@peninsulamalaysia.edu.my

Table of Contents

1.0 Introduction 6
1.1 Project Overview 6
1.2 Purpose and Objectives 7
1.3 Target Users and Stakeholders 7
1.4 Scope and Limitations 8
1.5 System Features and Components 9
2.0 Background 10
2.1 Problem Domain 10
2.2 Existing Issues and Limitations in Current Solutions 10
2.3 Motivation for Developing the System 11
2.4 Importance and Relevance of the System 12
2.5 Legal Considerations 12
2.6 Social and Ethical Considerations 13
2.7 Professional Responsibilities and Standards 13
2.8 Accessibility and Inclusivity Considerations 14
3.0 Project Management 15
3.1 Approach Adopted 15
3.2 Roles and Responsibilities 16
3.3 Communication Tools and Platform 17
3.3.1 Hardware Tools 17
3.3.2 Communication and Messaging Platform 17
3.3.3 File Sharing and Collaboration Platforms 18
3.4 Frequency and Format of Meetings 19
4.0 Development Approach 21

4.1 System Components 21
4.1.1 User Interface (UI) 21
4.1.2 Backend / Business Logic.....21
4.1.3 Database / Data Storage.....21
4.1.4 Integration Between Components 22
4.1.5 Development Tools and Testing 22
4.2 Technologies and Tools 24
4.2.1 Programming Languages 24
4.2.2 Frameworks and Libraries 24
4.3 Data Management.....27
4.3.1 Database Type and Structure 27
4.3.2 Data Integrity & Consistency Measures.....28
4.3.3 Data Security & Access Control 30
5.0 Requirements 31
5.1 Functional Requirements 31
5.1.1 Core System Features & Functionalities 31
5.1.2 User Interactions & System Behaviour 32
5.1.3 Data Input, Processing, and Output Functions.....33
5.2 Non-Functional Requirements 35
5.2.1 Security Requirements.....35
5.2.2 Performance & Reliability Requirements.....35
5.2.3 Usability & Accessibility Requirements 36
5.2.4 Scalability & Maintainability Considerations.....37
6.0 System Modeling 38
6.1 Use of UML / System Diagrams 38

6.2 Entity Relationship Diagram (ERD) 40
6.3 Use Case or User Story Representation 42
7.0 Sprint Planning / Project Scheduling 45
7.1 Project Timelines & Phase Planning 45
7.2 Task Breakdown, Dependencies & Prioritization 47
7.3 Sprint Review, Retrospective Activities, and Progress Monitoring 51
8.0 Software Engineering 55
8.1 Application of Software Engineering Principles 55
8.2 Modular & Reusable System Design 56
9.0 System Architecture 58
9.1 Overall Architectural Pattern 58
9.2 Interaction between System Components 59
9.3 Data Flow & Control Mechanisms 60
10.0 Design Diagrams 62
10.1 Class Diagrams 62
10.2 Sequence Diagrams 64
10.3 Activity Diagrams 66
10.4 State Diagrams 67
11.0 Code Quality & Standards 68
11.1 Coding Standards & Conventions 68
11.2 Version Control Practices 68
11.3 Code Review & Testing Procedures 69
11.4 Documentation & Maintainability Practices 69
12.0 Quality Assurance 70
12.1 Quality Assurance Strategy 70

12.2 Testing Planning & Execution....70
12.2.1 Unit Testing....70
12.2.2 Integration Testing....70
12.2.3 System & Acceptance Testing....71
References....72
Appendix A: GitHub Repository....73
Appendix B: Video Presentation....74
Appendix C: Design Diagrams....75
Appendix D: Project Timelines [Gantt Chart]....76

1.0 Introduction

1.1 Project Overview

In our campus, physical RFID access cards serve as the primary credentials for entering buildings, libraries, and laboratories. Due to staff and students forgetting, lost, or damaged cards, the system is prone to account lockout issues. This single point of failure caused numerous avoidable disruptions such as missed classes, late arrivals to exams, and administrative friction. As a result, the campus' efficiency and user experience will be ultimately undermined.

To address this issue, this project initially explored the feasibility of using smartphones to directly replace physical access control cards via Android Host Card Emulation (HCE) technology. However, practical experiments showed that direct communication with existing campus's Near Field Communication (NFC) access control readers was unsuccessful. This limitation stems from the fact that Android HCE only operates at the application layer, Application Protocol Data Unit (APDU) and cannot emulate underlying card attributes such as unique identifiers (UIDs) or proprietary authentication mechanisms required by commercial campus access control systems. Therefore, smartphones cannot truly replace physical cards when interacting with secure and reliable production-grade NFC readers.

In response to this technical constraint, the project scope was adjusted to focus on building a controlled, customized NFC authentication environment. Instead of emulating physical access cards, the system uses a smartphone as a secure NFC client. The smartphone application reads and stores the UID of a legitimate access card and sends this identifier as application data via APDU commands during HCE communication. A custom NFC reader, developed using an ESP32 microcontroller and a PN532 NFC module, receives this data, validates it against a predefined whitelist before granting or denying access.

1.2 Purpose and Objectives

Therefore, this project aims to develop a smartphone-based authentication prototype as a reliable backup mechanism to traditional card-based access systems. The system is designed to allow authorized users to use their personal smartphones when physical cards are unavailable, thereby reducing access disruptions while maintaining control and security at the card reader level.

This initiative follows clear SMART objectives:

- Develop an Android application capable of transmitting authenticated identity information via HCE.
- Design and implement a custom NFC reader using ESP32 and PN532.
- Ensure a measurable success rate in triggering access authorization signals.
- Ensure the solution is technically feasible within the Android NFC ecosystem.
- Deliver a fully functional proof-of-concept prototype by the project deadline (March 31, 2026)

The outcome of this project will be a practical and modern proof-of-concept system that demonstrates how smartphones can enhance access reliability without attempting to bypass or clone existing campus card infrastructure. By shifting authentication control to a custom reader environment, the solution balances feasibility, security, and usability, while providing a strong foundation for future expansion and integration.

1.3 Target Users and Stakeholders

The primary target users of the system are members of the campus community who require reliable access to secure facilities, particularly:

- Students, who depend on timely access to buildings, laboratories, lecture halls, libraries, examination venues, and so on.

1.4 Scope and Limitations

Scope of the Project:

The scope of this project focuses on the design and implementation of a prototype smartphone-based NFC authentication system operating within a controlled environment. Specifically, the project includes:

- Development of an Android mobile application utilising HCE
- Reading and secure storage of a legitimate access card UID within the mobile application
- Transmission of authentication data as application-level APDU messages via NFC
- Design and implementation of a custom NFC reader using ESP32 and PN532 hardware
- Reader-side validation of received identifiers against a predefined whitelist

Exclusions:

The project explicitly excludes the following:

- Direct emulation or replacement of official campus access cards
- Compatibility with existing production campus NFC access readers
- iOS platform support
- Full-scale campus deployment
- Integration with live institutional access control databases

Limitations:

Several constraints affect the project implementation:

- Time limitations imposed by the academic semester
- Hardware limitations associated with prototype-level NFC modules
- Platform dependency, as the solution relies on Android devices supporting NFC and HCE
- Security scope, as advanced cryptographic authentication mechanisms are beyond the project's intended proof-of-concept level

1.5 System Features and Components

Key System Features:

The proposed system provides the following core functionalities:

- Smartphone-based transmission of authenticated identifier data via NFC
- Secure application-layer communication using APDU commands
- Custom reader-side access validation based on whitelist matching
- Reliable fallback access mechanism when physical cards are unavailable

Main System Components:

The system consists of the following primary components:

- Mobile Application (Android)

  Provides the user interface, stores the card UID, and transmits authentication data using HCE

- Custom NFC Reader

  Implemented using an ESP32 microcontroller and PN532 NFC module to receive and process APDU messages

- Validation Logic

  Reader-side logic that compares received identifiers against stored authorised entries to determine access outcomes

2.0 Background

2.1 Problem Domain

The problem domain of this project lies within digital identity authentication and NFC-based access control systems. The environment involves a campus setting where users require fast, reliable, and secure authentication methods to ensure uninterrupted access to facilities. The system must operate in real-time, support multiple users, and maintain basic security standards appropriate for an academic prototype environment.

2.2 Existing Issues and Limitations in Current Solutions

Although RFID card systems are cost-effective and widely deployed, they present several limitations:

1. Dependence on Physical Cards
   Authentication is tied to possession of a physical object. If the card is forgotten, lost, or damaged, access is denied regardless of user identity.

2. Administrative Overhead
   Reissuing lost cards and manually granting temporary access increases workload for administrative staff.

3. Single Point of Failure
   The absence of a secondary authentication method makes the system inflexible.

Biometric systems such as fingerprint and facial recognition have been adopted in some institutions. However, these systems:

- Require expensive hardware infrastructure
- Raise concerns regarding biometric data storage
- Introducing privacy and regulatory compliance challenges
- May fail under certain environmental conditions

Additionally, experimentation during this project demonstrated that Android HCE cannot fully emulate proprietary campus access cards due to limitations in UID-level emulation and low-level protocol control. This reveals a technical gap between theoretical NFC capability and practical deployment in secure commercial systems.

2.3 Motivation for Developing the System

The motivation for this project arises from the need to design a practical and technically feasible backup authentication mechanism that:

- Reduces dependency on physical access cards
- Minimises access disruption
- Operates within realistic technical constraints
- Avoids bypassing or cloning existing secure systems

Rather than attempting to replace institutional infrastructure, this project aims to explore a controlled NFC authentication model that demonstrates how smartphones can function as secure identity clients within a customized validation environment.

The shift toward a custom reader architecture reflects a design decision based on experimental findings, aligning the project with achievable and academically sound objectives.

2.4 Importance and Relevance of the System

This system is relevant because:

- Smartphones are widely owned by students and staff.
- NFC technology is already mature and integrated into modern Android devices.
- Digital authentication aligns with broader trends in mobile identity systems (e.g., contactless payments and digital credentials).

By demonstrating a smartphone-based NFC authentication prototype, the project:

- Improves access reliability in situations where physical cards are unavailable.
- Encourages innovation within campus digital infrastructure.
- Provides a scalable foundation for future secure authentication research.

Although the prototype operates in a controlled environment, it reflects real-world authentication principles and contributes to understanding mobile-based identity systems.

2.5 Legal Considerations

The development of an authentication system requires awareness of legal and regulatory obligations, particularly concerning data handling and privacy.

1. Data Protection and Privacy

   Even though the prototype stores only card UID values rather than personal data, care must be taken to avoid misuse. In real deployments, compliance with personal data protection laws such as Malaysia's Personal Data Protection Act (PDPA) 2010 would be required.

2. Intellectual Property and Reverse Engineering

   The project does not attempt to reverse engineer or bypass proprietary campus access control systems. Instead, it operates within a self-developed test environment to avoid violating intellectual property or security regulations.

3. Security Compliance

Any future deployment would require alignment with institutional IT security policies and access control regulations.

2.6 Social and Ethical Considerations

Authentication systems directly impact user trust and privacy. Ethical considerations include:

- Avoiding unauthorized duplication of access credentials
- Ensuring data stored in the application is not exposed or misused
- Preventing unfair access privileges

The project intentionally avoids cloning official access cards and does not attempt to compromise existing campus security. By using a custom reader environment, the system maintains ethical boundaries while exploring authentication feasibility.

Additionally, user consent and transparency are essential principles in any identity-based system. Users must understand how their authentication data is stored and used.

2.7 Professional Responsibilities and Standards

As a computing project, the development process must adhere to professional standards, including:

- Ensuring system reliability and controlled testing
- Maintaining accurate documentation
- Conducting ethical experimentation
- Avoiding security exploitation

The project reflects responsible engineering practices by:

- Recognising technical limitations of HCE
- Avoiding illegal emulation attempts
- Designing a safe, academic test environment

2.8 Accessibility and Inclusivity Considerations

The system design considers accessibility in the following ways:

- The Android application interface should maintain readable text, clear feedback indicators, and simple navigation.
- NFC-based interaction requires minimal technical knowledge from users.
- The solution serves as a backup option, increasing inclusivity for users who may forget physical cards.

However, limitations exist:

- The system depends on Android devices supporting NFC and HCE.
- Users without compatible smartphones cannot use this backup mechanism.

Future improvements could include multi-platform support or alternative authentication channels to enhance inclusivity.

3.0 Project Management

3.1 Approach Adopted

In the NFCampus project, our team adopted a meticulously designed and adaptable meeting rhythm that balances the urgency of academic deadlines with the dynamic nature of agile development. This approach combines systematic rigor with human-centered care which ensures that every team member remains informed, engaged, and empowered throughout the project lifecycle.

We implemented a hybrid agile framework that tailored to the academic constraints and technical objectives. The framework divides the development into two-week iterations, and each is initiated by a sprint planning meeting. These meetings are critical and crucial for decomposing high-level objectives such as implementing HCE technology, designing secure credential storage modules, and establishing test reader communication channels into assignable, concrete tasks. This process ensures work consistently aligns with the project's SMART criteria, particularly focusing on the quantifiable goal of "successful card reading interactions".

We hold 15-minute stand-up meetings weekly to maintain momentum and transparency. These sessions are not progress reports to managers but team-led progress confirmations. Each member must briefly share their previous day's accomplishments, today's plans, and any obstacles encountered. This routine mechanism enables timely identification of technical bottlenecks such as NFC permission handling or simulation mode conflicts. Through this collaborative approach, issues can be resolved swiftly which prevents minor problems from escalating into major delays.

At the end of each sprint, we will hold a sprint review and retrospective meeting. The review meeting will focus on showcasing deliverables, validating them through actual testing, and providing feedback based on our progress. The subsequent retrospective meeting is the cornerstone of our people-centered approach. During the meetings, we discuss and share our work such as areas for improvement, and suggested process adjustments. This continuous feedback loop enables us to continually improve our workflows, communication methods, and technical strategies.

Other than agile processes, we implemented structured documentation and communication protocols. All technical decisions, API key integrations, and architecture diagrams are maintained on GitHub repository.

This approach enabled us to systematically address the complexities of NFC technology and Android development while maintaining team cohesion and motivation and ensuring we remained focused on delivering a fully functional and secure prototype by the March 31, 2026, deadline.

3.2 Roles and Responsibilities

(Table 3.0: Roles and Responsibilities)

Team Members	Roles & Responsibilities
Eu Jin Yang	● Project leadership and coordination ● Full-stack development ● Research and Documentation ● Developed application settings, preferences, and custom reader ● Team collaboration & support
Ng Kean Chun	● Designed application logo ● Developed the Image Capture and Optical Character Recognition (OCR) module ● Developed application profile tabs and QR Code log activities
Teoh Yi Shan	● Hardware & Testing Coordination ● Client Communication Lead & Meeting Minutes ● Project Documentation & Administrative Support
Yong Ken	● Unified Modeling Language (UML) & Technical Modelling ● System Architecture & Documentation ● Gantt Chart Development Lead ● Collaboration & Quality Assurance ● Designed poster

3.3 Communication Tools and Platform

Effective communication and collaboration are essential for coordinating tasks, managing development progress, and resolving technical challenges throughout the NFCampus project. To support these needs, a combination of hardware tools and software platforms was used to facilitate messaging, meetings, file sharing, testing, and task coordination.

3.3.1 Hardware Tools

Laptops served as the primary development and coordination devices for the project. They used to run Android Studio for application development and emulation, operate the Firebase console for backend configuration and database management, and review system designs and UI prototypes created using Figma. Laptops were also essential during meetings for live demonstrations, debugging sessions, and collaborative problem-solving.

Android mobile phones were used extensively for real-world testing and validation. Unlike emulators, physical devices allowed accurate testing of hardware-dependent features such as OCR, NFC, and HCE. Mobile phones were also used to test interactions with external hardware components, including the PN532 NFC module, ESP32 with expansion board, and LCD display, ensuring that the system behaved correctly under realistic usage conditions.

3.3.2 Communication and Messaging Platform

WhatsApp was used as the primary communication and messaging platform for the team. It supported daily communication, coordination of tasks, scheduling of meetings, and rapid discussion of technical issues. Daily stand-up updates, reminders, and progress confirmations were shared through the WhatsApp group chat, enabling continuous visibility of each member's contributions.

WhatsApp also functioned as the team's main documentation and reference channel. Key decisions, action items, screenshots, links, and technical references were recorded in real time, creating a persistent and chronological communication log. This "always-on" progress timeline

allowed team members to revisit previous discussions, track decisions, and maintain continuity across different phases of the project.

The selection of WhatsApp was based on both practical and contextual considerations. It is a platform deeply integrated into the team members' daily communication habits, enabling instant and convenient interaction with minimal learning overhead. Its compatibility across mobile devices and laptops ensured accessibility regardless of location or device availability, making it suitable for both on-campus and remote collaboration.

3.3.3 File Sharing and Collaboration Platforms

GitHub was used as the primary platform for file sharing, version control, and technical collaboration. All source code, documentation, and configuration files were maintained in a centralized repository, allowing team members to work on different modules concurrently while tracking changes and preventing conflicts.

Figma was used as a collaborative design platform for UI and interface planning. It enabled team members to share design mock-ups, provide feedback, and refine layouts before implementation, improving consistency between design and development.

3.4 Frequency and Format of Meetings

Our team would gather at a quiet study lounge or the campus laboratory every Monday afternoon to have weekly face-to-face meetings. These weekly meetings are more than just simple attendance checks. They are strategic workshops where new ideas are sparked, code comes alive on the screen, and the collective clarity of thought stems from individual efforts.

Structure and Flow:

The meetings typically begin with a relaxed yet focused roundtable discussion. Each member ready with their laptop, prepares notes, and shares the following:

a. Progress has been made since the last meeting,
b. Any obstacles encountered, whether technical, logistical, or conceptual,
c. And their immediate goals for the following week.

This was followed by a live demonstration session, where the members are responsible for a recently developed feature such as OCR integration, the NFC binding interface, or the HCE service to share their screen to demonstrate each function to the team. This practice not only fostered accountability but also served as an informal peer review where feedback could be instantly incorporated and brainstorming for improvements.

Daily Stand-ups:

In the spirit of Agile-Scrum, we adopted a daily stand-up ritual that was held every weekday evening. Unlike weekly meetings, these meetings are conducted entirely online via WhatsApp, adapting to different schedules and locations while maintaining the rhythm of daily accountability.

Format and Discipline:

Every evening at a fixed time, the WhatsApp group would turn from casual into a formal update zone. Each team member would post a short message which follows a simple three-part template:

1. What did I do today? (e.g., “Completed the MFA login flow integration,” “Debugged NFC UID capture issue”)
2. What do I plan to do tomorrow? (e.g., “Start HCE service implementation,” “Write unit tests for OCR module”)

3. Any blockers or help needed? (e.g., “Stuck with Firebase Auth rules, can someone review?”)

These updates were expected to be concise, specific, and transparent. We intentionally avoid lengthy discussions in the main post while detailed problem-solving will be arranged in sub-posts or ad-hoc calls.

Ad-hoc and Milestone Meetings:

The constant evolution of projects often requires impromptu collaboration while our fixed weekly and daily rhythm provides stability. We hold ad-hoc meetings whenever critical vulnerabilities arise, design adjustments are needed, or integration issues occur between front-end and back-end modules. These meetings are typically organized via quick WhatsApp polls (“Can we have a meeting in 30 minutes?”) and can take the form of voice chat, video calls, or impromptu face-to-face discussions.

Furthermore, we hold longer working meetings before key milestones such as proposal submissions, mid-term demos, or final demos. These meetings are more intensive, which often last several hours, and focus on integration testing, slide preparation, or report finalization. During these sessions, the roles of programmers, writers, designers, and testers flexibly switch, reflecting the cross-functional spirit of our team.

4.0 Development Approach

4.1 System Components

4.1.1 User Interface (UI)

The user interface is built using Jetpack Compose, Android's modern declarative toolkit. UI architecture follows a state-driven approach to ensure an easy-to-use and accessible design. Key UI modules include:

- Registration Flow: A multi-step process implemented to ensure data integrity.
- Dynamic Dashboard: A home screen featuring a real-time generated QR code for external verification and a "Recent Activity" feed.
- Security Management: Dedicated screens for account termination, email / password updates, and NFC troubleshooting.

4.1.2 Backend / Business Logic

The backend logic is distributed between the mobile client and a lightweight external server:

- AuthViewModel: Manages the authentication state machine, including email verification, login session persistence, and hardware-level security checks.

- HCECardService: A background service extending HostApduService that handles low-level NFC ISO-DEP communication, allowing the phone to emulate a smart card.

- Node.js Server: A secondary backend (Express.js) hosted locally that handles the QR code verification requests, fetching student logs from Firestore to display on a web interface.

4.1.3 Database / Data Storage

The system employs two types of storage data: Firebase Firestore and SharedPreferences. Firebase is used as a sort of cloud storage. It is primarily used to create or verify authentication of users through the login and registration, storing user profiles, device binding IDs, and activity logs. And when used with Node.js Server, it can be used to access specific user profiles from the database.

The reason Firebase has chosen to be our cloud storage is due to the great security infrastructure it provides. Firebase Authentication integrates seamlessly with Firestore Security Rules, allowing us to restrict data access on a per-user basis strictly. This ensures that users can only access their own profiles and device binding IDs.

On the other hand, SharedPreference is a local storage that stores the campus card UID and application preferences locally to allow for near-instant NFC responses without requiring a network round-trip during door-tapping.

The decision to divide data storage between cloud and local sources is primarily a security measure. By storing the campus card UID exclusively in SharedPreferences, we ensure that the card's UID is never transmitted over the network or stored in the cloud. Therefore, even if a malicious actor compromises a user's login credentials, they cannot access the physical UID needed to emulate the campus card.

4.1.4 Integration Between Components

Integration is achieved through Firebase SDKs and Reactive Streams (StateFlow). The MainActivity acts as the integration hub, synchronizing the Firestore data with local storage upon login. For external hardware integration, the system uses the Android NFC Foreground Dispatch to bridge physical card IDs into the digital Firestore environment.

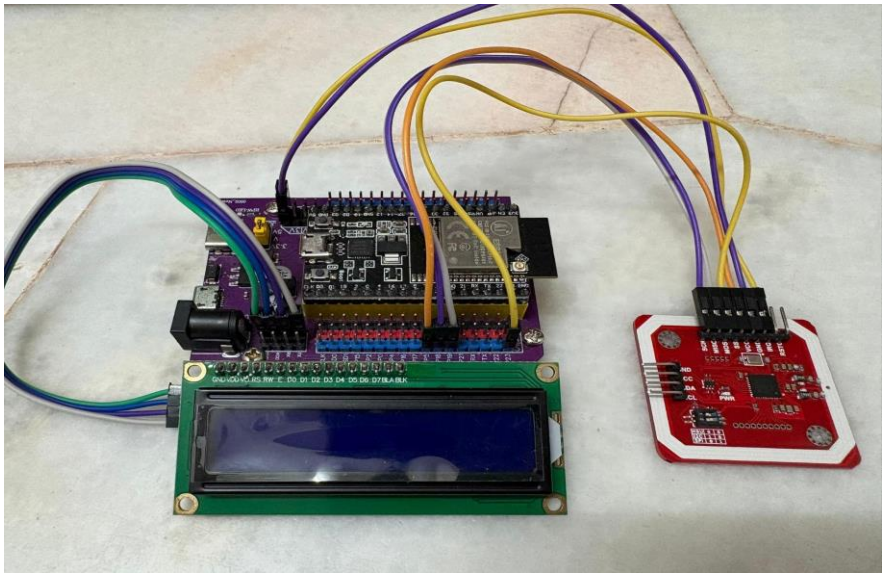
4.1.5 Development Tools and Testing

The development and validation of the system use a combination of Integrated Development Environments (IDEs), cloud-based monitoring tools, and hardware-level testing. Android Studio is used as the primary IDE, allowing us to write and compile the Kotlin / Java codebase. Its built-in Logcat utility was needed for real-time debugging, allowing us to monitor system logs, trace application execution flow, and diagnose runtime exceptions during the development cycle.

To manage and observe the cloud infrastructure, the Firebase Console was utilized. This web-based interface enabled the team to monitor the Firestore database in real time, verify the integrity

and results of incoming activity logs, and dynamically configure the security rules that govern data access control, ensuring that user data remained protected throughout testing.

Due to the specific hardware requirements of the application, Physical Device Testing was required during the validation phase. Therefore, all functional testing for proximity sensing and text recognition was conducted on physical Android hardware to validate usability, accuracy, and reliability under various environmental conditions.



(Table 4.0: Prototype Device Assembled to Simulate Real-World Door Access)

To simulate a real-world door access environment for the tests, a hardware prototype was assembled consisting of a NodeMCU ESP32 microcontroller paired with a PN532 NFC / RFID Kit to act as the NFC reader. A display was also integrated into the setup to provide visual feedback during testing such as displaying UID matches or access denial messages. This physical testbed allowed the team to validate the end-to-end flow of HCE-based communication, ensuring that the Android application could successfully emulate a card and communicate with the reader under realistic conditions.

4.2 Technologies and Tools

4.2.1 Programming Languages

The development of the system consists of multiple programming languages to optimize both client-side performance and server-side reliability.

- Kotlin

  Selected as the primary language for the Android application due to its modern syntax, type safety, and full interoperability with Java. Kotlin's concise nature reduced boilerplate code while maintaining readability. A key technical advantage of Kotlin was the use of Coroutines for managing asynchronous operations. This allowed the application to perform network calls to Firebase and background tasks without blocking the main UI thread, ensuring a smooth user experience during login, registration, and activity log synchronization.

- JavaScript (Node.js)

  Utilized to build the lightweight backend verification server. JavaScript, running on the Node.js runtime, was chosen for its event-driven, non-blocking architecture, which is ideal for handling multiple concurrent verification requests from the Android clients. The server acts as the middleman, using the Firebase Admin SDK to securely access specific user profiles and device binding data from Firestore for validation purposes.

- JSON (JavaScript Object Notation)

  Employed as the primary data interchange format throughout the system. JSON's lightweight and human-readable structure made it ideal for configuring application settings and structuring the data payloads exchanged between the Android client, the Node.js server, and Firebase Firestore. Its native support in both Kotlin and JavaScript ensured compatible parsing and serialization across all layers of the application.

4.2.2 Frameworks and Libraries

The Android application was built upon a modern technology stack, leveraging several key libraries and frameworks to deliver a responsive user experience.

- Jetpack Compose

  Employed as the modern UI toolkit for building the application's interface. Allowing the development of complex UI components and reduced the amount of boilerplate code required for rendering user profiles, activity logs, and authentication screens.

- Google ML Kit

  Integrated to provide OCR capabilities for extracting student data from physical identification cards. Utilizing Google's on-device text recognition APIs, the application can accurately capture and digitize information such as student names, identification numbers, and other printed details directly from the camera feed.

- CameraX

  Utilized as the camera interface library to manage the complexities of interacting with the device's camera hardware. CameraX provides a consistent and lifecycle-aware API that works across most Android devices, eliminating the need to write device-specific code. It was particularly critical for the OCR functionality, as it provided a smooth preview feed and image analysis pipeline for the ML Kit text recognizer.

```kotlin
QRCodeGenerator.kt   NFCSetupStep.kt   ForgotPasswordScreen

package com.example.nfcampus.gui.components

import android.graphics.Bitmap
import android.graphics.Color
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel

2 Usages
fun generateQRCodeBitmap(data: String): ImageBitmap {
    val size = 1020 // Define the size of the QR code image

    // Set up QR code hints
    val hints = mapOf(
        EncodeHintType.ERROR_CORRECTION to ErrorCorrectionLevel.H,
        EncodeHintType.MARGIN to 1 // Set a small margin
    )

    val bitMatrix = QRCodeWriter().encode(
        contents = data,
        format = BarcodeFormat.QR_CODE,
        width = size,
        height = size,
        hints
```

(Figure 4.1: Using ZXing Import for QRCode Generation)

- ZXing (Zebra Crossing)

  Figure 4.2.2 shows ZXing imports for QRCode Generation. Implemented to generate high-density QR codes for student identity verification. When a user's identity needs to be verified, the application encodes their profile information into a secure, scannable QR code.

- Coil (Coroutine Image Loader)

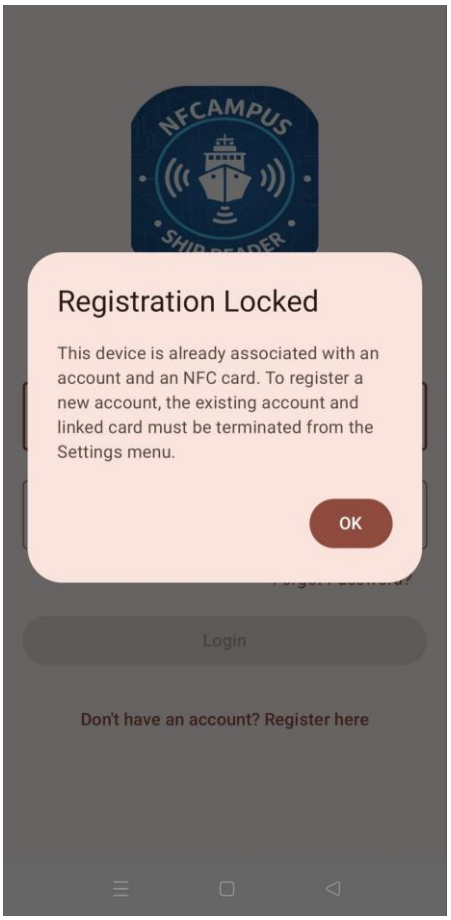
  Selected for efficient, asynchronous image loading throughout the application. Coil was specifically chosen for its native integration with Kotlin Coroutines, seamlessly fitting into the application's existing asynchronous architecture. It handles the fetching, caching, and displaying of student ID photos from Firebase Storage, ensuring that profile images load quickly and smoothly without blocking the UI thread or consuming excessive memory.

4.3 Data Management

4.3.1 Database Type and Structure

The system employs Cloud Firestore, a flexible, scalable NoSQL document database provided by Firebase. Firestore was selected for its real-time synchronization capabilities, robust security features, and seamless integration with the Firebase Authentication system. The database scheme is structured to optimize both data security and query performance:


```
backImageUri: "file:///data/user/0/com.example.nfcampus/cache/mlkit_
deviceId: "7bb021049a437012"
email: "ngkeanchun120005@gmail.com"
frontImageUri: "file:///data/user/0/com.example.nfcampus/cache/mlkit
fullName: "Ng Kean Chun"
identificationNumber: "050612020957"
intake: "June 2025"
major: "BSc (Hons) Computer Science (Software-Engineering)"
nfcUid: "04:A3:B2:C1"
password: ""
studentId: "BSSE2506036"
uid: "UvuhQSoGN5STd84LVOslA6GsFJV2"
```

(Figure 4.2: Collection Stores User Profile and “One Device, One Account” Policy)

- users Collection

  Figure 4.2 shows the users’ collection. This collection stores individual user profiles, with each document identified by the unique Firebase Auth UID generated during registration. Each user document contains essential profile information such as student name, email,

user ID, and Majors. It also stores the deviceId, which is used to enforce the “One Device, One Account” security policy by binding the user account to a specific physical device.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // Rule for the 'users' collection
    match /users/{userId} {
      // Allow user to manage their own profile
      allow read, create, update, delete: if request.auth != null && request.auth.uid == userId
    }

    // Rule for the 'activity_logs' collection

    match /activity_logs/{userId} {
      allow read, write, delete: if request.auth != null && request.auth.uid == userId;

      match /entries/{logId} {
        allow read, write, delete: if request.auth != null && request.auth.uid == userId;
      }
    }
  }
}
```

(Figure 4.3: Rules for database that help segregated specific data from others)

- activity_logs Collection

  Figure 4.3 shows the activity_logs collection. To maintain data isolation and optimize query performance, user data and activity logs are stored using a nested collection structure. For example: activity_logs/{userId}/entries/{logId}. This hierarchical approach ensures that each user’s access history and profile are logically segregated from others, preventing unauthorized cross-user data exposure. This structure also simplifies data lifecycle management, as deleting a user document can cascade to their nested log entries, ensuring complete removal of user data when accounts are terminated.

4.3.2 Data Integrity & Consistency Measures

To maintain data integrity and prevent the accumulation of orphaned records or “ghost data”, the system implements several defensive mechanisms:

```
LinkedCardScreen.kt   NFCTroubleshootingScreen.kt   ChangeEmailPasswordScreen.kt   ProfileScreen.kt   MainS
35   fun LinkedCardScreen(
71       Scaffold { _ ->
256          Dialog(onDismissRequest = { showTerminateDialog = false }) {
262              ) {
268                  ) {
336              // 1. Delete the ENTRIES sub-collection in activity_logs
337              Log.d( tag = "TerminateDebug",  msg = "2. Fetching logs from activity_logs/$userId/entries")
338              db.collection( collectionPath = "activity_logs")
339                  .document( documentPath = userId)
340                  .collection( collectionPath = "entries")
341                  .get()
342                  .addOnSuccessListener { snapshot ->
343                      val batch = db.batch()
344                      for (doc in snapshot) {
345                          batch.delete( documentRef = doc.reference)
346                      }
347
348                      // 2. Commit logs, then delete the userId document in activity_logs, then users
349                      batch.commit().addOnCompleteListener {
350                          Log.d( tag = "TerminateDebug",  msg = "3. Logs wiped. Deleting activity_logs doc and user profile.")
351
352                          // Delete the parent doc in activity_logs
353                          db.collection( collectionPath = "activity_logs").document( documentPath = userId).delete()
354
355                          // Delete the main user profile
356                          db.collection( collectionPath = "users").document( documentPath = userId).delete()
357                              .addOnCompleteListener {
358                                  Log.d( tag = "TerminateDebug",  msg = "4. Firestore cleaned. Deleting Auth.")
359
360                                  // 3. Delete Auth Account
361                                  currentUser.delete().addOnCompleteListener { task ->
362                                      // 4. SharedPreference/Local Cleanup
363                                      activity?.let { act ->
364                                          act.getSharedPreferences( name = "nfcampus_prefs",  mode = Context.MODE_PRIVATE)
```

(Figure 4.4: Atomic Batch Deletions Implemented within the Logic for the "Terminate Linked Card")

- Atomic Batch Deletions

  Figure 4.4 shows the implementation of Atomic Batch Deletions. During account termination, the system utilizes Firestore's WriteBatch functionality to execute multiple delete operations as a single atomic transaction. This ensures that either all user-related data, including the main user document and all nested activity log entries is deleted successfully, or none of it is. This approach prevents partial deletions that could leave orphaned log entries referencing non-existent user profiles.

- Lifecycle-Aware Synchronization

  The application implements a foreground detection mechanism that triggers the syncUidFromFirestoreToPreferences function whenever the app returns to the active state. This lifecycle-aware synchronization ensures that the locally stored campus card UID in SharedPreferences always matches the authoritative version stored in Firestore. By reconciling data at every app resume, the system prevents inconsistencies caused by background updates or multi-device access scenarios.

4.3.3 Data Security & Access Control

The system enforces a comprehensive security model centered around the principle of "One Device, One Account", implemented through multiple complementary layers:

- Hardware Binding

  During the registration process, the application captures the device's unique deviceId and stores it within the user's Firestore document. The AuthViewModel implements a verification check during login that compares the current device's deviceId against the stored value. On the other hand, the NFC Card linked into the device also has its own unique uid, by checking campus_card_uid in sharedPreferences and device Id in Firestore, we implemented a dual-layer verification mechanism that binds both the physical device and the NFC card to a single user identity. Any attempt to register a new account when the account and card uid is already binded to this device is immediately blocked, preventing NFC card overwrite and credential sharing.

- Firestore Security Rules

  At the database level, Firebase's declarative security rules enforce strict access control. Rules are configured to verify that a user's requests.auth.uid matches the document ID they are attempting to read or write. This approach ensures that authenticated users can only access their own profile data and activity logs, preventing scenarios where one user could inadvertently or maliciously access another's records.

- Account Termination Cleanup

  When a user initiates account deletion, the system executes a comprehensive cleanup routine. Beyond deleting cloud data via atomic batch operations, the application also triggers a recursive deletion of all locally stored data in SharedPreferences, including the sensitive campus card UID. This ensures that after account termination, no residual NFC credentials remain on the physical device that could be exploited for unauthorized door access, providing complete data sanitization at both cloud and local levels.

5.0 Requirements

The system's requirements are categorized into functional requirements, which define the specific behaviors and capabilities the system must exhibit, and non-functional requirements, which establish the quality attributes, performance benchmarks, and constraints governing system operation.

5.1 Functional Requirements

5.1.1 Core System Features & Functionalities

- Identity Extraction (OCR)

  The system must utilize the device's rear-facing camera to capture images of physical student identification cards. Through integration with Google ML Kit's OCR framework, the system shall automatically extract and parse critical student data fields, including the Student Name, Identification Number (Matriculation ID), and Course of Study. The extracted data must be presented to the user for verification prior to account registration to ensure accuracy.

- Hardware Binding

  The system must enforce a strict "One Device, One Account" security policy by linking each user account to a unique physical device identifier. During registration, the application shall capture the device's deviceId and store it within the user's Firestore document. A dual-validation mechanism is implemented, checking both the bound device identifier and the locally stored campus card UID to prevent multiple accounts from being associated with a single device or card. Any attempt to register a new account using a device or card already bound to an existing user shall be immediately rejected.

- NFC Identity Emulation (HCE)

  The mobile application must function as a virtual smart card capable of emulating a physical campus identification card. Utilizing the HCE framework, the system will transmit a stored UID to a custom NFC reader using the ISO-DEP (ISO 7816-4) protocol. The HCE

service must remain active while the application is in the foreground, responding to polling requests from compatible reader hardware without requiring user intervention.

- Activity Auditing

  The system must maintain a comprehensive audit trail of user actions to support security monitoring and forensic analysis. Timestamped logs shall be recorded for all critical operations, including user registration, login attempts, and door access attempts. These logs must be stored securely in Firestore within a nested collection structure activity_logs/{userId}/entries/{logId} to ensure data isolation and facilitate efficient querying.

- Account Termination

  The system must provide a mechanism for users to permanently terminate their account and unlink their device. Upon termination, the system will execute a recursive deletion process that atomically removes all cloud-stored user data, including the user document and all associated activity log entries while simultaneously wiping local credentials, including the campus card UID stored in SharedPreferences. This ensures complete data sanitization and prevents residual credentials from being exploited after account termination.

5.1.2 User Interactions & System Behaviour

- Registration Gate

  To ensure the validity of user accounts, the system must enforce email verification as a mandatory prerequisite. Following initial registration input, the system will then send a verification link to the provided email address. Users may only proceed to the NFC card setup phase after successfully verifying their email address, ensuring that all registered accounts are associated with legitimate, accessible email addresses.

- HCE Activation

  The HCE service must automatically activate and respond to compatible NFC readers whenever the application is in the foreground and a valid UID is stored in

SharedPreferences. Upon detecting a polling reader, the system transmits the stored UID without requiring additional user input.

- Dynamic UI State

  The application's Home screen must provide real-time visual feedback reflecting the current system's status. The interface will display an "Active" or "Inactive" indicator for the virtual card, update dynamically based on the presence of a valid UID, and generate a unique, scannable QR code encoding the user's identification data for manual inspection scenarios where NFC is unavailable.

5.1.3 Data Input, Processing, and Output Functions

- Input

  The system will accept data from multiple sources, including:

  - Hardware Sensors: Camera input for OCR-based identity extraction and NFC antenna input for card reading during registration.

  - User Text Input: Manual entry of credentials during login and registration, including email address and password**.**

- Processing

  The system must perform several distinct data processing operations:

  - Text Parsing: OCR results will be filtered through a configurable list of restricted keywords to remove unnecessary text information from the card and ensure the accuracy of extracted student data.

  - QR Encoding: User identification data, including User ID and Unix timestamps, shall be encoded into high-density bitmap images using the ZXing encoding algorithm for generation of scannable identity QR codes.

- APDU Handling: The HCE service must process APDU received from NFC readers and respond with the appropriate 14-byte response containing the stored card UID, conforming to the ISO 7816-4 specification.

- Output

  The system will provide multimodal feedback to users:

  - Haptic Feedback: Vibration patterns that indicate successful versus failed NFC transactions.

  - Visual Dialogs: Success and error dialogs should appear for critical operations such as login failures and account termination.

  - Web-Based Verification: The Node.js server will generate web-based verification pages accessible to administrators for validating user identities and reviewing access logs.

5.2 Non-Functional Requirements

5.2.1 Security Requirements

- Data Isolation

  Firestore security rules must be configured to enforce strict data segregation at the database level. Users will only be permitted to read, write, or delete documents within their own users document and their dedicated activity_logs/{userId}/entries sub-collection. All rule evaluations must verify that request.auth.uid matches the target document's user identifier before granting access.

- Atomic Cleanup

  Account deletion operations must be executed atomically to prevent partial data removal. The system will utilize Firestore's WriteBatch functionality to group all deletion operations, including the user document and all nested activity log entries into a single atomic transaction. If any individual deletion fails or a network interruption occurs, the entire batch operation must be rolled back, ensuring no orphaned "ghost data" remains in the database.

5.2.2 Performance & Reliability Requirements

- NFC Latency

  The HCE service must respond to reader challenges within a maximum latency of 500 milliseconds. This threshold is critical to ensuring a smooth, responsive "tap-and-go" experience at door access points, preventing user frustration caused by delayed or failed access attempts due to timeout conditions.

- Persistence

  The campus card UID must be stored in persistent local memory using SharedPreferences to ensure core NFC functionality remains available even under poor or absent internet connectivity. Users shall be able to perform door access operations regardless of network conditions with cloud synchronization occurring asynchronously when connectivity is restored.

- Sync Reliability

  The application must perform a synchronization operation, invoking the syncUidFromFirestoreToPreferences function every time the application returns to the foreground. This ensures that the locally stored device state remains consistent with the authoritative Firestore record, preventing inconsistencies that could arise from background updates, account modifications, or cross-device administrative actions.

5.2.3 Usability & Accessibility Requirements

- Feedback Mechanisms

  The system must provide multimodal feedback to ensure users clearly understand the outcome of their actions. Distinct haptic vibration patterns shall differentiate successful transactions from failed attempts, while visual indicators such as green checkmarks for success and red error icons for failures shall provide immediate, glanceable feedback on the application interface.

- Design Consistency

  The user interface must adhere to Material 3 (Material You) design standards established by Google, ensuring visual consistency with the broader Android ecosystem. The design incorporates high-contrast text for improved readability, intuitive navigation icons, and responsive layouts that adapt to varying screen sizes and orientations.

- Guidance

  To reduce user friction during hardware interactions, the application shall include an integrated “NFC Troubleshooting” module. This module will provide visual diagrams and step-by-step instructions guiding users on correct phone placement relative to campus NFC readers, addressing common issues such as positioning, case interference, and antenna location.

5.2.4 Scalability & Maintainability Considerations

- NoSQL Data Architecture

  The use of Cloud Firestore as a NoSQL document database provides inherent scalability advantages over traditional relational databases. The system can accommodate growth from dozens to thousands of users without requiring manual database schema migrations or complex sharding strategies, as Firestore automatically partitions and distributes data as demand increases.

- Sub-collection Organization

  The decision to store activity logs within a nested entries sub-collection under each user document helps with maintainability. This hierarchical structure enables faster targeted data wipes during account termination compared to a single collection, simplifies query construction, and naturally enforces data isolation at the database level.

- Modular Component Design

  The Android application follows modular architecture with clear separation of concerns. Logic is organized into distinct Repositories and ViewModels. This separation ensures that future developers can update individual components such as replacing the OCR engine or modifying the NFC protocol implementation without requiring extensive rewrites of the core application logic, significantly reducing maintenance overhead and technical debt.

6.0 System Modeling

System modeling provides a visual and structural representation of the NFCampus system, illustrating how different components interact and how data flows through the application. This section presents various UML diagrams and models that define the system's architecture, behavior, and data relationships.

6.1 Use of UML / System Diagrams

The NFCampus project employs UML diagrams to model both static and dynamic aspects of the system. UML provides a standardized way to visualize the system's design, making it easier for stakeholders and developers to understand the system's structure and behavior.

The key UML diagrams utilized in this project include:

- Use Case Diagrams
  Illustrate the functional requirements and interactions between users and the system. They define what the system should do from the user's perspective.

- Class Diagrams
  Represent the static structure of the system, showing classes, their attributes, methods, and relationships. This provides a blueprint for object-oriented implementation.

- Sequence Diagrams
  Displaying the dynamic interaction between objects over time, particularly useful for understanding the NFC authentication flow and Firebase integration.

- Activity Diagrams
  Model the workflow and business logic, showing the step-by-step process from user registration to NFC-based access validation.

- State Diagrams

Capture the different states of key components such as the authentication process and NFC service lifecycle.

These diagrams collectively provide a comprehensive view of the system, facilitating better communication among team members and ensuring alignment between design and implementation.

6.2 Entity Relationship Diagram (ERD)

The ERD defines the data model and relationships within the NFCampus system. The system primarily uses Firestore as a NoSQL database, which stores data in collections and documents rather than traditional relational tables.

Key Entities:

1. User
    - Attributes: userID (Primary Key), email, displayName, createdAt, lastLogin
    - Description: Represents registered users of the system

2. Card
    - Attributes: cardID (Primary Key), userID (Foreign Key), cardUID, registeredAt, status
    - Description: Stores NFC card information linked to users
    - Constraint: One-to-one relationship with User ("One Device, One Account")

3. AccessLog
    - Attributes: logID (Primary Key), userID (Foreign Key), cardUID, timestamp, accessGranted, location
    - Description: Record all access attempts for audit and monitoring purposes

Relationships:

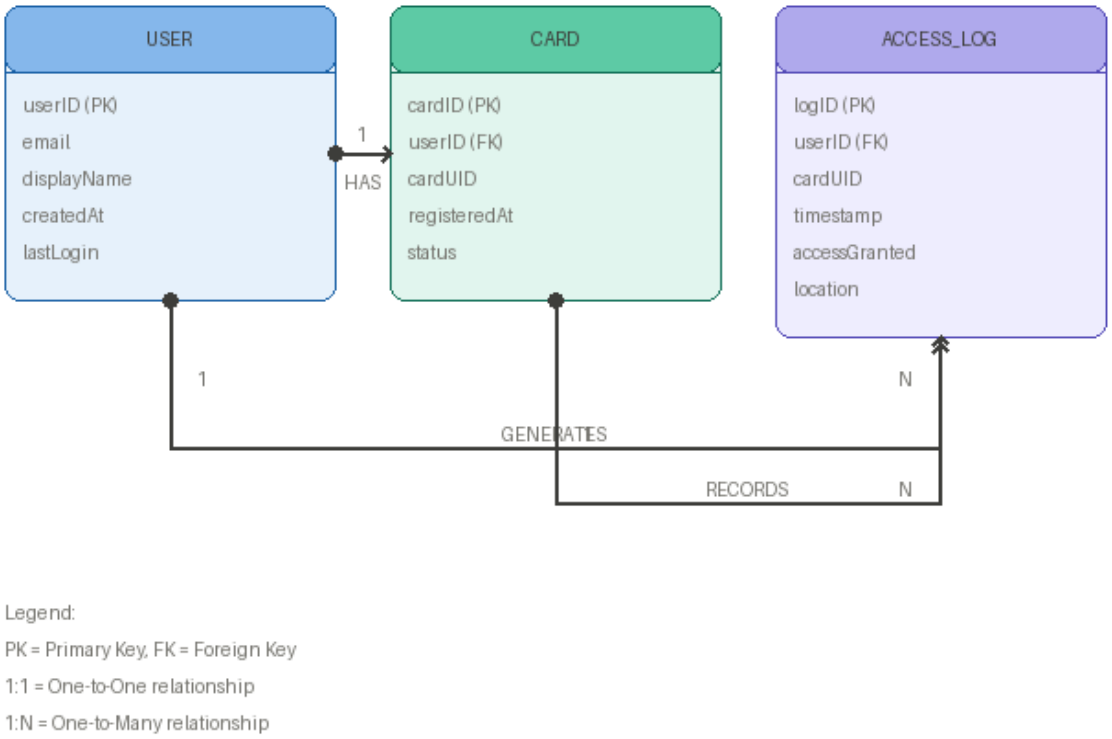
- User to Card: One-to-One relationship. Each user can register only one card at a time, enforcing the business rule of single card per account.
- User to AccessLog: One-to-Many relationship. A user can have multiple access log entries over time.
- Card to AccessLog: One-to-Many relationship. Each card can be associated with multiple access attempts.

The ERD ensures data integrity through the following mechanisms:

- Unique constraints on email addresses and card UIDs

- Cascade deletes rules to maintain referential integrity
- Indexed fields for efficient querying (userID, cardUID, timestamp)

While Firestore is a NoSQL database, the ERD concept is adapted to represent the document structure and references between collections, maintaining logical relationships through document references and subcollections.



(Figure 6.0: Entity Relationship Diagram)

6.3 Use Case or User Story Representation

The system is designed around several key use cases that define the primary interactions between users and the NFCampus application.

Primary Use Cases:

UC-01: User Registration

- Actor: New User
- Description: A new user creates an account using email and password authentication
- Preconditions: User has the NFCampus app installed and internet connectivity
- Main Flow:
    1. User opens the app and selects “Register here”
    2. User enters email and password
    3. System validates input and creates account in Firebase Authentication
    4. System sends email verification link
    5. User verifies email through the link
    6. System confirms registration and grants access
- Postconditions: User account is created and email is verified

UC-02: User Login

- Actor: Registered User
- Description: An existing user logs into the application
- Preconditions: User has a registered account with verified email
- Main Flow:
    1. User enters email and password
    2. System authenticates credentials via Firebase
    3. System retrieves user profile and card data from Firestore
    4. System grants access to main dashboard
- Postconditions: User is authenticated and can access app features

UC-03: Card Registration via OCR

- Actor: Authenticated User

- Description: User registers their physical NFC card by capturing its details (text) using the device camera and OCR
- Preconditions: User is logged in and has not registered a card yet
- Main Flow:
    1. User tap “Scan Front / Back of Card” option
    2. User positions physical card in front of camera
    3. System captures image and processes it using OCR
    4. System extracts card details (text) from the image
    5. User confirms the extracted details (text)
    6. System stores details (text) in Firestore and SharedPreferences
    7. System displays confirmation message
- Postconditions: Details (text) are linked to the user’s account

UC-04: NFC-Based Access Authentication

- Actor: Authenticated User with Registered Card
- Description: User authenticates at an access point using smartphone NFC
- Preconditions: User has registered card details (text), HCE service is active, NFC is enabled
- Main Flow:
    1. User approaches NFC readers with smartphone
    2. NFC reader initiates communication
    3. Smartphone’s HCE service receives APDU command
    4. HCE service retrieves stored card UID
    5. HCE service transmits card UID to ESP32 via PN532 reader
    6. ESP32 validates card UID against whitelist
    7. Reader grants or denies access
    8. System logs access attempt
    9. User receives feedback (visual / audio)
- Postconditions: Access is granted or denied, and attempt is logged

UC-05: View Access History

- Actor: Authenticated User

- Description: User views their past access attempts and history
- Preconditions: User is logged in
- Main Flow:
    1. User navigates to “Home” section and scan QR Code
    2. System retrieves access logs from Firestore
    3. System displays chronological list of access attempts
    4. User can filter by date or status
- Postconditions: User views their access history

User Stories (Agile Format):

- As a campus user, I want to register my account using email, so that I can securely access the system.
- As a registered user, I want to easily capture my card details (text) using my phone’s camera, so that I don’t have to manually type long text.
- As a user who forgot my physical card, I want to use my smartphone for NFC authentication, so that I can still access buildings without disruption.
- As a system administrator, I want to enforce “One Device, One Account”, so that security and accountability are maintained.
- As a user, I want to view my access history, so that I can track when and where I accessed facilities.

These use cases and user stories guided the development process and ensured that the system meets real user needs while maintaining security and usability standards.

7.0 Sprint Planning / Project Scheduling

The NFCampus project follows an Agile-Scrum methodology with iterative development cycles. This section outlines the sprint structure, task breakdown, and progress monitoring mechanisms employed throughout the project lifecycle.

7.1 Project Timelines & Phase Planning

The project is organized into distinct phases spanning from September 2025 to April 2026, totaling approximately 24 weeks of development. The timeline is structured to align with academic milestones and ensure steady progress toward the final deliverable.

Project Timelines [Gantt Chart] - See Appendix D, pg.76

Phase 1: Planning & Requirements (Weeks 1 – 4, September 2025)
This initial phase establishes the foundation for the entire project:

- Early Development and Proof of Concept (POC) to validate technical feasibility
- Requirement Analysis to identify stakeholder needs and system constraints
- Literature Review to understand existing solutions and identify gaps
- Scope Definition and methodology selection (Agile-Scrum framework)

Phase 2: Design & Architecture (Weeks 5 – 8, October 2025)
Focus shifts to system design and initial implementation:

- System Design & Planning including high-level architecture and database schema
- Frontend development begins with UI / UX design for login and registration screens
- OCR integration for card text extraction
- Authentication module implementation using Firebase
- Proposal writing and interim submission preparation

Phase 3: Core Development (Weeks 9 – 16, November 2025 – January 2026)
The main development sprint where critical features are implemented:

- NFC and HCE service integration for card emulation

- Backend API development for user management and UID validation
- Database operations and “One Device, One Account” enforcement
- Integration of frontend, backend, and NFC hardware components

Phase 4: Testing & Refinement (Weeks 17 – 20, February – March 2026)

Quality assurance and system validation:

- Comprehensive testing (Unit, Integration, and System testing)
- Bug identification and resolution
- Usability testing with real devices
- Performance optimization

Phase 5: Documentation & Finalization (Weeks 21 – 24, March – April 2026)

Project closure and deliverable preparation:

- Complete documentation including user guides and technical specifications
- Final report compilation with diagrams, screenshots, and analysis
- Presentation preparation including slide deck and system demonstration
- Final submission and project handover

This phased approach ensures systematic progress while allowing flexibility to adapt to challenges and feedback throughout the development process.

7.2 Task Breakdown, Dependencies & Prioritization

Tasks are organized into sprints with clear dependencies and priority levels to ensure efficient workflow and timely completion.

Sprint Structure:
The project follows 2-week sprint cycles, each concluding with a sprint review and retrospective. This cadence allows for regular assessment and course correction while maintaining momentum.

Sprint 1 – 2 (Weeks 1 – 4): Foundation
High Priority Tasks:

- Set up development environment (Android Studio, Firebase, GitHub repository)
- Conduct stakeholder interviews and gather functional requirements
- Create initial system architecture document
- Develop proof-of-concept for NFC UID reading

Dependencies:

- Requirements gathering must be completed before design can begin
- Development environment setup is a prerequisite for all coding tasks

Sprint 3 – 4 (Weeks 5 – 8): Authentication & UI
High Priority Tasks:

- Design and implement user registration and login UI
- Integrate Firebase Authentication for email / password authentication
- Implement email verification workflow
- Develop OCR module for card text extraction
- Design database schema in Firestore

Medium Priority Tasks:

- Create user profile management screens
- Implement input validation and error handling

Dependencies:

- Firebase setup must be completed before authentication features can be tested
- UI design should align with finalized system requirements
- OCR testing requires physical cards and camera-enabled devices

Sprint 5 – 6 (Weeks 9 – 12): NFC Integration

High Priority Tasks:

- Implement Android HCE service for NFC communication
- Develop UID storage mechanism using SharedPreferences and Firestore
- Create custom NFC reader using ESP32 and PN532 module
- Establish APDU command protocol between phone and reader
- Implement UID validation logic on reader side

Critical Dependencies:

- HCE service depends on successful UID storage implementation
- Reader hardware must be functional before end-to-end testing can occur
- Firebase rules must be configured to support “One Device, One Account” constraint

Sprint 7 – 8 (Weeks 13 – 16): Backend & Business Logic

High Priority Tasks:

- Develop user management API endpoints
- Implement “One Device, One Account” enforcement logic
- Create access logging system
- Build whitelist management for authorized UIDs
- Integrate all system components (App, Firebase, Hardware)

Medium Priority Tasks:

- Implement access history retrieval and display
- Add user notifications for successful/failed access attempts

Dependencies:

- Backend APIs require database schema to be finalized
- Integration testing depends on completion of all major modules

Sprint 9 – 10 (Weeks 17 – 20): Testing & Quality Assurance

High Priority Tasks:

- Conduct unit testing for all major components
- Perform integration testing between app and Firebase
- Execute system testing with actual hardware
- Conduct usability testing with target users
- Document and fix identified bugs

Medium Priority Tasks:

- Performance testing under various network conditions
- Security testing for authentication and data transmission
- Edge case testing such as network failures and invalid inputs

Dependencies:

- All core features must be implemented before comprehensive testing
- Bug fixes may require revisiting earlier sprint deliverables

Sprint 11 – 12 (Weeks 21 – 24): Documentation & Finalization

High Priority Tasks:

- Complete final project report with all required sections
- Create comprehensive system documentation
- Prepare presentation slides and demo video
- Rehearse final presentation
- Submit all project deliverables

Medium Priority Tasks:

- Update GitHub repository with clean code and README
- Create user manual and troubleshooting guide

Dependencies:

- Report writing requires completion of all development and testing phases
- Presentation cannot be finalized until system is fully functional

Prioritization Framework:

Tasks are prioritized using the MoSCoW method:

- Must Have: Core functionality (Authentication, NFC Communication, UID Validation)
- Should Have: Important features that enhance usability (OCR, Access History)
- Could Have: Nice-to-have features if time permits (Advanced Analytics)
- Won’t Have: Features deferred to future iterations (Multi-card Support, Biometric Integration)

This prioritization ensures that critical features are delivered first, minimizing risk and ensuring a functional system even if time constraints arise.

7.3 Sprint Review, Retrospective Activities, and Progress Monitoring

Each sprint concludes with structured review and retrospective sessions to evaluate progress, identify improvements, and plan the next iteration.

Sprint Review Process:

At the end of each 2-week sprint, the team conducts a sprint review meeting to:

1. Demonstrate Completed Work

   Show functional increments to stakeholders or supervisors such as working authentication flows, successful NFC reads, or integrated backend APIs.

2. Validate Against Requirements

   Compare delivered features against the sprint goals and acceptance criteria defined at sprint planning.

3. Gather Feedback

   Collect input on usability, functionality, and potential improvements from reviewers.

4. Update Product Backlog

   Adjust priorities based on feedback and emerging requirements.

Sprint Retrospective Activities:

Following the sprint review, the team holds a retrospective to reflect on the process itself:

1. What Went Well

   Identify successes such as smooth Firebase integration, successful hardware assembly, or effective problem-solving approaches.

2. What Didn’t Go Well

   Discuss challenges like OCR accuracy issues, NFC communication bugs, or time management difficulties.

3. Action Items

Define specific improvements for the next sprint, such as adopting better testing practices, improving documentation, or scheduling more frequent check-ins.

4. Process Adjustments

   Modify workflows based on lessons learned such as increasing buffer time for hardware debugging or allocating more time for research when encountering unfamiliar technologies.

Progress Monitoring Mechanisms:

The project employs several tools and practices to track progress and ensure timely delivery:

1. Burndown Charts

   Visual representation of remaining work versus time, helping the team assess whether they are on track to meet sprint goals.

2. Task Board (Kanban Style)

   Using tools like Trello, GitHub Projects, or physical boards to track tasks through stages:

   - To Do
   - In Progress
   - In Review
   - Done

3. Weekly Stand-ups

   Brief synchronous or asynchronous updates where team members share:

   - What they accomplished since the last update
   - What they plan to work on next
   - Any blockers or issues requiring assistance

4. GitHub Commit Activity

   Monitoring repository activity to ensure consistent code contributions and identify potential bottlenecks.

5. Milestone Tracking
   Key milestones such as “Authentication Complete”, “NFC Integration Working”, and “All Tests Passing” serve as major checkpoints.

6. Risk Register
   Maintained throughout the project to track potential risks such as hardware compatibility issues and API limitations, and mitigation strategies.

Metrics Tracked:

- Velocity: Story points or tasks completed per sprint, helping predict future capacity
- Defect Density: Number of bugs found per module, indicating code quality
- Test Coverage: Percentage of code covered by automated tests
- Sprint Goal Achievement Rate: Percentage of planned work completed each sprint

Adaptive Planning:

The Agile approach allows for adaptive planning. When the team discovered that direct HCE emulation wouldn’t work with campus readers, the sprint plan was adjusted to:

- Pivot from card emulation to custom reader development
- Allocate additional time for ESP32 and PN532 integration
- Defer some UI enhancements to accommodate hardware development

This flexibility ensured the project remained viable despite technical challenges, demonstrating the value of iterative development and regular review cycles.

Communication and Collaboration:

Progress monitoring is supported by:

- Regular supervisor meetings for guidance and feedback
- Team communication via messaging platforms such as WhatsApp
- Shared documentation using Google Docs for collaborative writing
- Version control through Git for code management and collaboration

By maintaining disciplined sprint reviews, honest retrospectives, and comprehensive progress monitoring, the NFCampus project stays on track while continuously improving both the product and the development process.

8.0 Software Engineering

This section details the software engineering principles and design strategies applied to ensure the system is robust, maintainable, and scalable. The development of this prototype emphasizes a disciplined approach to managing complexity and promoting code reusability.

8.1 Application of Software Engineering Principles

The development of this smartphone-based NFC authentication system is guided by several core software engineering principles to ensure a high-quality, reliable, and maintainable prototype.

- Separation of Concerns

  The Android application separates the UI logic from the business logic and data access layers. For instance, the HCECardService class handles NFC communication independently of the MainActivity, which manages user interactions. This separation ensures that changes to the UI, such as updating the dynamic interface, do not inadvertently affect the core NFC emulation logic. Similarly, the reader-side firmware on the ESP32 separates hardware control (PN532) from access control logic.

- Defensive Programming

  The system incorporates defensive programming to handle errors and unexpected states gracefully. Input validation is performed at multiple levels. For example, during registration, the system validates user input in email format and OCR-extracted data before proceeding. The Node.js server validates all incoming API requests to prevent malformed data from corrupting the Firestore database. On the reader side, the ESP32 firmware includes error handling for failed NFC communication, ensuring the system remains stable and does not enter an undefined state.

- Secure by Design

  Security is not an afterthought but is integrated into the foundational design. The “One Device, One Account” policy, enforced through hardware binding, is a prime example. This principle prevents unauthorized account proliferation. The use of Firestore Security

Rules to control data access, the mandatory email verification gate, and the complete data sanitization upon account termination are all manifestations of a security-first mindset.

- Version Control and Documentation
  The project utilizes Git for version control, enabling systematic tracking of changes, feature branching, and collaborative development. Comprehensive documentation is maintained both in-code using comments to explain complex logic such as APDU command processing and in external documents to ensure knowledge transfer and future maintainability.

8.2 Modular & Reusable System Design

The system is decomposed into well-defined, loosely coupled modules to manage complexity and enhance reusability.

Android Application Modules:

- OCR Module: Encapsulates the integration with Google ML Kit. This module provides a clean interface (extractStudentData(Bitmap image)) to capture an image and return a structured data object. This abstraction allows the core registration logic to remain unchanged even if the underlying OCR library were to be replaced.

- HCE Module: The HCECardService class is a self-contained module that handles all aspects of NFC communication. It manages the lifecycle of the HCE service, processes incoming APDU commands, and transmits the stored UID. This module can be easily integrated into other Android projects requiring HCE functionality.

- Data Storage Module: A dedicated module using a Repository pattern abstracts the data source from the rest of the app. It determines whether to read / write from local SharedPreferences for UID storage or from the remote Firestore database for user accounts and logs. This promotes reusability and simplifies testing.

Backend (Firebase & Node.js) Modules:

- Firestore Data Models: The database is structured with clear, reusable collections (users, activity_logs). The use of a nested collection structure for logs activity_logs/{userId}/entries is a modular design that isolates user data, making it more secure and efficient to query.

- Node.js Serverless Functions: The web-based verification page is served by a Node.js function. This function is a modular component that can be extended to provide an API for other administrative tools in the future, separate from the main application logic.

Reader Firmware Modules:

- NFC Communication Module: A C++ class encapsulates all interactions with the PN532 module, providing simple functions like readCardUID() and sendAuthenticationResponse(). This abstraction hides the complexities of low-level I2C / SPI communication and makes the main access control logic cleaner and more readable.

- Access Control Module: This module manages the whitelist that is stored in EEPROM or a local database on the ESP32 and contains the core logic for comparing a received UID against the list and triggering the appropriate output such as activating a relay for door control. This module is independent of the hardware interface, allowing the same logic to be used with different types of readers or output mechanisms.

9.0 System Architecture

This section describes the high-level structure of the system, detailing its architectural pattern, component interactions, and data flow. The architecture is designed to support the functional requirements while adhering to the defined constraints.

9.1 Overall Architectural Pattern

The system employs Client-Server Architecture with a hybrid model that incorporates both cloud-based and embedded components.

- Android Client

  The Android application acts as the primary client. It is a fat client, handling significant business logic such as OCR processing, local data storage for unique identifiers like the card UID and user session credentials, and real-time UI updates. Its interaction with the backend server is primarily for cloud data synchronization during registration, login, and logging activities, as well as for performing operations that require a trusted environment such as email verification.

- Backend Server (Firebase & Node.js)

  Firebase serves as the backend-as-a-service platform, providing authentication through Firebase Auth and a NoSQL cloud database via Firestore. A lightweight Node.js server is used to host a web-based verification page, which acts as an administrative client. This hybrid approach leverages the rapid development benefits of Firebase while providing custom server-side logic for specific administrative needs.

- Embedded System as a Service

  The custom NFC reader implemented using an ESP32 microcontroller and PN532 module functions as a standalone service. It does not directly interact with the backend server. Instead, it operates as an independent access control unit that relies on a locally stored whitelist of authorized identifiers. This architecture choice ensures the core fallback mechanism for phone-to-reader authentication remains functional even without an internet connection, thereby addressing a key reliability requirement.

9.2 Interaction between System Components

The interactions between the four main components which are Android App, Firebase Backend, Node.js Server, and Custom NFC Reader are defined by specific sequences.

- Registration & Setup:
    1. The Android App captures a student ID card image.
    2. It uses the OCR Module to extract student data.
    3. The app sends the user's email, password, and extracted data to Firebase Authentication and Firestore to create a new user account.
    4. Firebase sends a verification email. Upon verification, the app proceeds to the card setup phase.
    5. The app reads a physical campus card's UID using the device's NFC reader.
    6. It binds this UID and the device's deviceId to the user's document in Firestore and stores the UID locally in SharedPreferences.

- Authentication & Access:
    1. A user with the Android App in the foreground brings their phone near the Custom NFC Reader.
    2. The Custom NFC Reader (PN532 / ESP32) polls for a device and sends an APDU command.
    3. The Android App's HCE Service responds with an APDU containing the stored UID.
    4. The Custom NFC Reader compares the received UID against its local whitelist.
    5. If a match is found, the reader triggers the access mechanism and logs the attempt locally.
    6. The Android App provides haptic feedback to the user to indicate success or failure.

- Activity Auditing:
    1. After a successful login or access attempt from the app side, the Android App sends a log entry to Firestore, storing it in the activity_logs/{userId}/entries collection.

The access attempt logged in the app is based on the user's action of opening the app. The actual door access event is logged by the reader.

- Account Termination:
  1. A user initiates account termination from within the Android App.
  2. The app calls a multi-step process to recursively delete the user's document and all associated activity logs from Firestore.
  3. Upon successful cloud deletion, the app clears the locally stored UID and user data from SharedPreferences.

9.3 Data Flow & Control Mechanisms

The flow of data and control through the system is managed by specific mechanisms to ensure security, consistency, and responsiveness.

- Data Flow:
  - Registration: Camera → OCR Module → UI (for User Verification) → Firebase Auth & Firestore (Cloud) & SharedPreferences (Local)
  - Authentication: NFC Reader (PN532) → HCE Service (APDU Request) → HCE Service (Retrieves UID from SharedPreferences) → NFC Reader (APDU Response)
  - Auditing: User Action (such as Login) → Android App → Firestore (activity_logs Collection)
  - Access History: Admin → Node.js Server (Requests Page) → Firestore (Queries User Data) → Node.js Server (Generates HTML) → QR Code
- Control Mechanisms:
  - Asynchronous Callbacks

The Android application heavily utilizes asynchronous callbacks for all cloud interactions (Firebase) and long-running operations (OCR). This prevents the UI thread from being blocked, ensuring a smooth and responsive user experience.

- Event-Driven Control on Reader

  The ESP32 firmware is event-driven. The loop() function continuously checks for events such as nfc.inListPassiveTarget() events indicating a phone is present. Upon this event, it triggers the APDU command / response sequence and the subsequent access control logic. This is an efficient pattern for embedded systems that spend most of their time waiting for external inputs.

- State Management

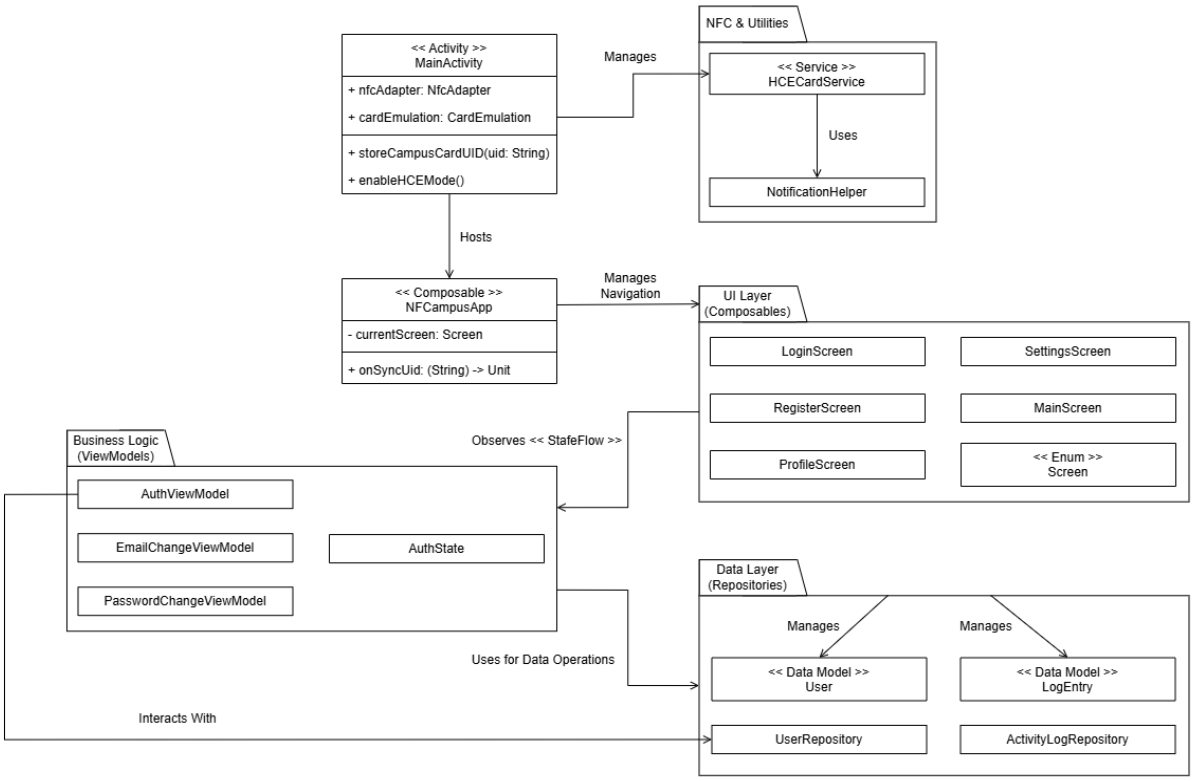
  The Android application manages its state to control the HCE service. The HCE service is only active when the user is logged in and a valid UID is stored in SharedPreferences. This state is managed by the MainActivity and communicated to the HCECardService via a shared SharedPreferences listener or a bound service mechanism, ensuring the system is only vulnerable to emulation when it is intended to be.

- Firestore Security Rules

  Firestore Security Rules act as a gatekeeper whereby allowing read / write access only to authenticated users and only to their own documents. This server-side control is a critical mechanism for ensuring data integrity and security, regardless of the client's behavior.

10.0 Design Diagrams

10.1 Class Diagrams



(Figure 10.0: Class Diagrams)

GitHub Link [Class Diagrams] - See Appendix C, pg.75

Figure 10.0 shows the structural design of the NFCampus system, illustrating the relationships between different components in the application layer.

MainActivity acts as the central controller, managing NFC functionality such as HCE mode and UID storage, and hosting NFCampusApp, which handles UI navigation and data synchronization.

The system uses layered architecture:

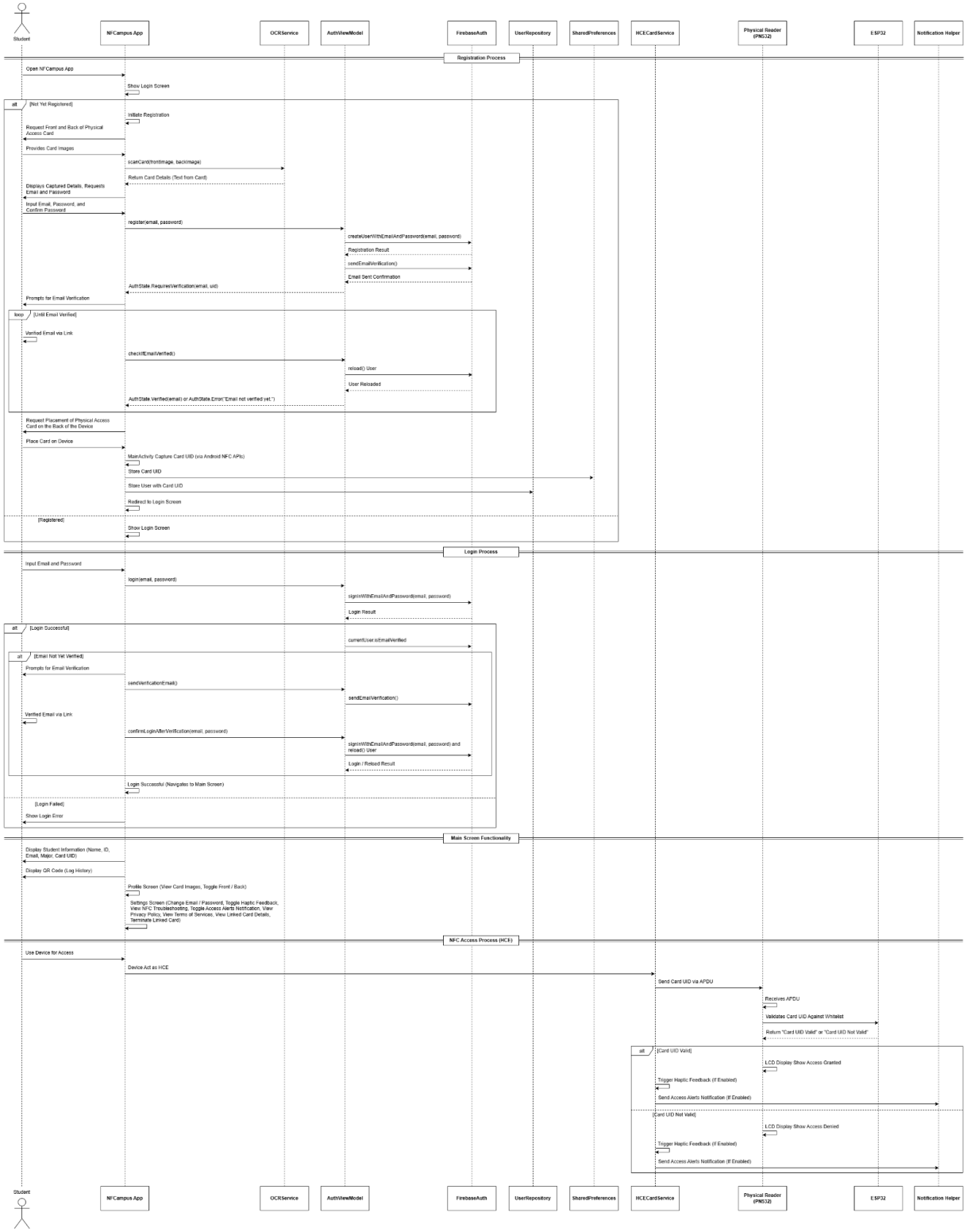
- UI Layer: Composable screens such as LoginScreen, RegisterScreen, MainScreen, ProfileScreen, and SettingsScreen for user interaction.

- Business Logic Layer: ViewModels such as AuthViewModel, EmailChangeViewModel, and PasswordChangeViewModel manage logic, authentication state (AuthState), and connect the UI and data via StateFlow.

- Data Layer: Repositories such as UserRepository and ActivityLogRepository handle data operations and models such as User and LogEntry with Firebase and local storage.

HCECardService manages NFC communication, sending UID via APDU and triggering notifications via NotificationHelper.

10.2 Sequence Diagrams



(Figure 10.1: Sequence Diagrams)

GitHub Link [Sequence Diagrams] - See Appendix C, pg.75

Figure 10.1 shows the sequence of interactions between system components during registration, login, and NFC access.

Students open the app and are taken to the login screen. Unregistered users start registration. The app uses OCR to scan the physical card and extract details, verifying the email address and password before submission.

AuthViewModel communicates with Firebase to create an account and send a verification email. The system checks if the email address is verified before proceeding. Once verified, the user places the physical card on the device. MainActivity captures the UID via the Android NFC API and stores it locally in SharedPreferences and UserRepository.

During login, credentials are verified via Firebase. If the email address is not verified, the system prompts again. After successful login, the user is taken to the main screen.

For NFC access, the device acts as an HCE card via HCECardService. When the device is tapped on the PN532 card reader, the UID is sent via APDU. The ESP32 validates the UID against a whitelist and determines access permissions.

Feedback is provided via LCD display, haptic feedback, and optional notifications.

10.3 Activity Diagrams

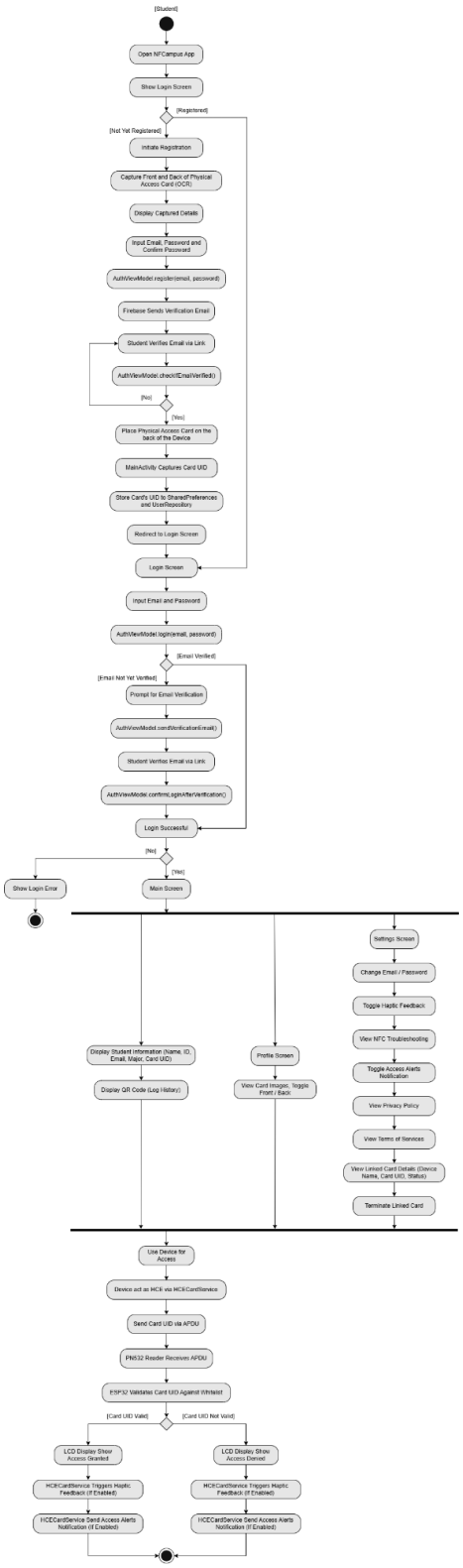
GitHub Link [Activity Diagrams] - See Appendix C, pg.75

Figure 10.2 shows the overall workflow of the NFCampus system. Starting from user authentication to NFC-based access control.

Students will see a login screen when opening the application. Unregistered users proceed to registration. They capture the front and back of the physical access card via OCR technology, review the information, and then enter their email address and password. After registration, Firebase will send a verification email. Users must verify their email address. Once verified, users place their physical card on the device to capture its UID, which is stored in SharedPreferences and the user repository.

The user then logs in. If the email address is not verified, the system will prompt for verification again. After successful login and verification, the user can reach the main screen. From the main screen, users can view their profile, view QR code logs, or manage account settings such as change email / password, enable notifications, and terminate linked cards.

For access control, users tap the device on the NFC reader. The HCE service sends stored UID via APDU. ESP32 will verify the UID against a whitelist. If the UID is valid, access is granted. Otherwise, access is denied. Feedback is provided via LCD screen, haptic vibration, and optional notifications.



(Figure 10.2: Activity Diagrams)

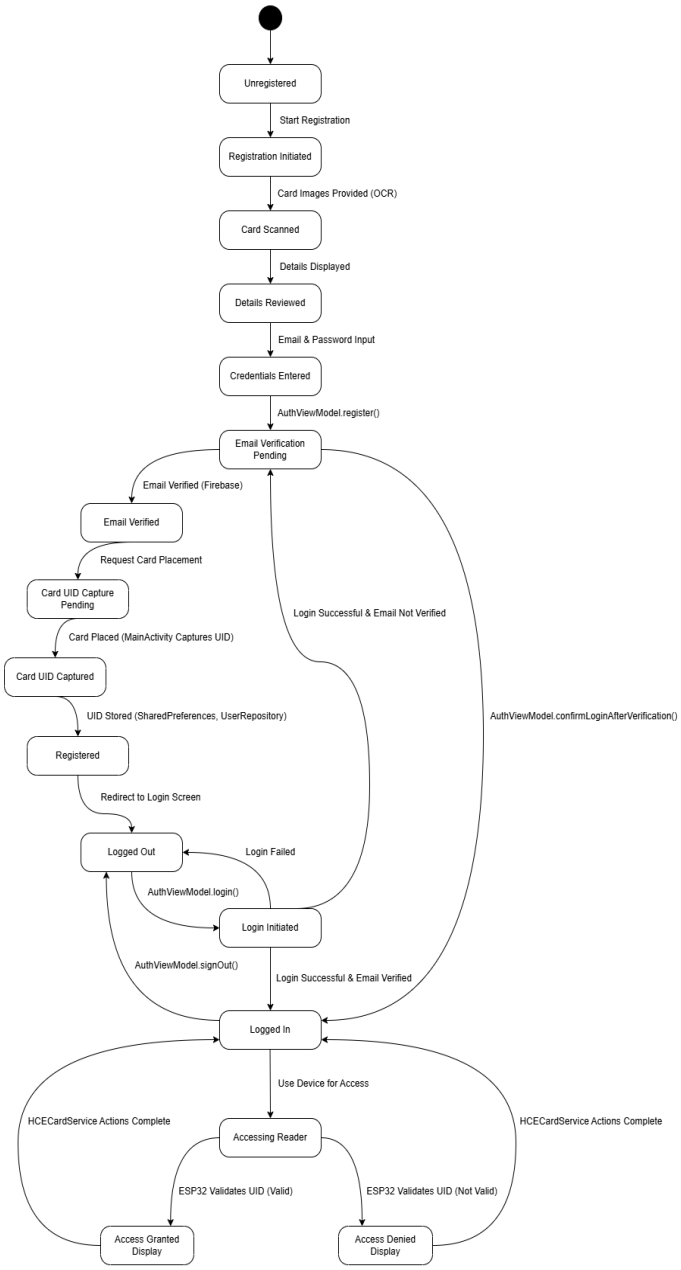
10.4 State Diagrams

[GitHub Link [State Diagrams] - See Appendix C, pg.75]

Figure 10.3 shows the different states of the NFCampus system and the transitions between them during user interaction.

The system initially appears as "Unregistered". When registration, it sequentially progresses through 3 states: "Registration Initiated", "Card Scanned", and "Credentials Entered". After submission, it enters the "Email Verification Pending" state.

Once email verification is successful, the system enters "Email Verified" state and captures the card UID. After being stored, it enters the "Registered" state and redirects to login. During login, the system sequentially progresses through the "Logged Out" and "Login Initiated" states. On failure, it returns to "Logged Out" state. On success with verified email, it enters "Logged In" state.



(Figure 10.3: State Diagrams)

In the "Logged In" state, the user performs NFC access. When interacting with the reader, the system enters the "Accessing Reader" state. Based on UID validation, it sequentially displays either "Access Granted" or "Access Denied" before returning to the "Logged In" state.

11.0 Code Quality & Standards

11.1 Coding Standards & Conventions

The NFCampus project follows consistent coding standards to ensure readability, maintainability, and scalability. Kotlin was used as the primary programming language and strictly follows the official Kotlin style guidelines. Naming conventions such as camelCase for variables and functions, and PascalCase for classes were consistently applied.

Code is structured using a modular architecture, separating concerns into ViewModels, repositories, and UI components. This ensures that business logic, data handling, and presentation layers remain independent and easy to manage.

The codebase extensively uses proper indentation, meaningful variable names, and inline comments to improve code clarity and reduce ambiguity.

11.2 Version Control Practices

GitHub is used as a version control system for managing source code and documentation. The team follows a structured workflow, with each feature or bug fix developed in separate branches before being merged into the main branch.

Commit messages are clear, concise, and accurately reflecting the changes made. This practice improves code traceability and allows team members to understand the project's evolution.

Regular commits and synchronization ensure minimal merge conflicts and maintain code consistency across the team.

11.3 Code Review & Testing Procedures

Code reviews were conducted informally during weekly meetings and through peer discussions. Team members reviewed each other's code to identify potential issues, improve logic, and ensure compliance with coding standards.

Testing procedures were integrated into development where features were tested immediately after implementation. Debugging tools such as Logcat in Android Studio were used to monitor runtime behavior and identify errors.

This continuous review and testing approach helps maintain code reliability and reduce the incidence of defects.

11.4 Documentation & Maintainability Practices

Throughout the project, comprehensive documentation was maintained including code comments, UML diagrams, and system design descriptions. Key modules such as AuthViewModel and HCECardService were documented to explain their responsibilities and interaction.

The use of modular design improves maintainability by allowing individual components to be updated or replaced without affecting the entire system. Furthermore, consistent documentation ensures that future developers could easily understand and extend the system.

12.0 Quality Assurance

12.1 Quality Assurance Strategy

The NFCampus project employs a continuous quality assurance approach, integrating testing and validation throughout the entire development lifecycle. Quality assurance focuses on ensuring the system's reliability, correctness, and usability.

Both software and hardware components are tested in controlled environments to simulate real-world usage. This strategy emphasizes early defect detection, iterative improvements, and validation based on system requirements.

12.2 Testing Planning & Execution

The testing was conducted in multiple stages to ensure that the system was fully validated.

12.2.1 Unit Testing

Unit testing focuses on validating individual components of the system such as authentication logic, OCR processing, and data storage functionality.

Key modules tested include:

- AuthViewModel (login, registration, email verification)
- SharedPreferences (UID storage and retrieval)
- OCR data extraction functions

These tests ensure that each component functions correctly in isolation before integration.

12.2.2 Integration Testing

Integration testing evaluates how different components interact with each other. This includes:

- Interaction between the mobile application and Firebase Authentication
- Synchronization between Firestore and SharedPreferences
- Communication between the HCE service and the NFC reader hardware

These tests ensure that data flows correctly between components and that combined functionalities operate as expected.

12.2.3 System & Acceptance Testing

System testing validates the entire application workflow from registration to NFC access. Real devices were used to test hardware-related functions such as OCR and NFC communication.

Acceptance testing was conducted to ensure the system meets user requirements, including:

- Successful registration and login flow
- Accurate card UID capture and storage
- Reliable NFC-based access validation
- Proper feedback for access granted or denied

The tests were considered successful when the system consistently performed all core functionalities under real-world conditions.

Word Count: 11,364

References

Ampheo. “How to Connect the NFC RFID PN532 V3 Reader Module to Arduino?” *Medium*, December 5, 2025. https://medium.com/@pqshedy33/how-to-connect-the-nfc-rfid-pn532-v3-reader-module-to-arduino-08b83d8c64fc.

Android Developers. “CameraX Overview | Android Media.” Android Developers, March 4, 2025. https://developer.android.com/media/camera/camerax.

Android Developers. “Host-Based Card Emulation Overview | Connectivity.” Android Developers, February 26, 2026. https://developer.android.com/develop/connectivity/nfc/hce.

Android Developers. “NFC Basics | Connectivity.” Android Developers, October 29, 2025. https://developer.android.com/develop/connectivity/nfc/nfc.

Arduino, Adafruit. “LiquidCrystal | Arduino Documentation.” August 10, 2017. https://docs.arduino.cc/libraries/liquidcrystal/.

Coil. “Coil.” 2024. https://coil-kt.github.io/coil/.

Google Developers. “Recognize Text in Images with ML Kit on Android.” Google for Developers, March 23, 2026. https://developers.google.com/ml-kit/vision/text-recognition/v2/android.

MyJPJ. *MyJPJ - Portal Rasmi JPJ*. April 11, 2025. https://www.jpj.gov.my/myjpj/.

Olli. “Using Host Card Emulation with Adafruit PN532 Reader - Azuki.” March 2, 2023. https://azukidigital.com/blog/hce.

Scott. “Generate QR Code Using ZXing Library.” *Medium*, December 27, 2022. https://medium.com/@copy2sim/generate-qr-code-using-zxing-library-751ca3d76792.

Appendix A: GitHub Repository

GitHub Link: https://github.com/JustJin6/NFCampus.git

Appendix B: Video Presentation

YouTube Link: https://youtu.be/fZcIQbSTMfs

Appendix C: Design Diagrams

Back to 10.1 Class Diagrams, pg.62

GitHub Link [Class Diagrams]:
https://github.com/JustJin6/NFCampus/blob/df9e3283637e4cd7997eb4983b4a728e0ebaf88f/Diagrams/NFCampus%20(Class%20Diagram).png

Back to 10.2 Sequence Diagrams, pg.64

GitHub Link [Sequence Diagrams]:
https://github.com/JustJin6/NFCampus/blob/df9e3283637e4cd7997eb4983b4a728e0ebaf88f/Diagrams/NFCampus%20(Sequence%20Diagram).png

Back to 10.3 Activity Diagrams, pg.66

GitHub Link [Activity Diagrams]:
https://github.com/JustJin6/NFCampus/blob/df9e3283637e4cd7997eb4983b4a728e0ebaf88f/Diagrams/NFCampus%20(Activity%20Diagram).png

Back to 10.4 State Diagrams, pg.67

GitHub Link [State Diagrams]:
https://github.com/JustJin6/NFCampus/blob/df9e3283637e4cd7997eb4983b4a728e0ebaf88f/Diagrams/NFCampus%20(State%20Diagram).png

Appendix D: Project Timelines [Gantt Chart]

Back to 7.1 Project Timelines & Phase Planning, pg.45

2025			Sept			Oct			Nov			Dec		
Phase	Task	Description	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
1	Development Environment Setup	Set up Android Studio, Firebase, GitHub repository												
	Requirement Analysis & Stakeholder Interviews	Identify problems, gather requirements, stakeholder interviews												
	Literature Review & Background Study	Survey related works, summarize strengths / weaknesses												
	Proof of Concept (POC) & Initial Architecture	Validate technical feasibility, create initial system architecture document												
2	System Design & Database Schema Planning	High-level architecture, database schema, interface planning												
	Frontend UI / UX Design (Login & Registration)	Design and implement login / registration UI screens												
	Firebase Authentication Module	Integrate Firebase Auth, email verification workflow												
	OCR Integration for Card Details Extraction	Develop OCR module for card details (text) extraction												

3	NFC & HCE Service Implementation	Implement Android HCE service for NFC communication												
	UID Storage (SharedPreferences & Firestore)	Develop UID storage mechanism and synchronization												
	Custom NFC Reader (ESP32 & PN532)	Design and implement custom NFC reader hardware												
	Backend API & Database Operations	Develop user management API, One Device, One Account enforcement												
	Proposal Writing & Interim Submission	Draft, review with supervisor, finalize, and submit proposal.												

2026			Jan			Feb			Mar			Apr		
Phase	Task	Description	W13	W14	W15	W16	W17	W18	W19	W20	W21	W22	W23	W24
3	APDU Protocol & Reader Validation Logic	Establish APDU command protocol, implement UID validation on reader												
	System Component Integration	Integrate app, Firebase, hardware, and backend components												
4	Unit Testing	Test individual components (Auth, OCR, Data Storage)												

	Integration Testing	Test component interactions, Firebase sync, HCE communication												
	System & Usability Testing	End-to-end testing, real device testing, usability validation												
	Bug Fixing & Performance Optimization	Identify and resolve bugs, optimize performance												
5	Complete Documentation	User guides, technical specifications, code documentation												
	Final Report Compilation	Compile report with diagrams, screenshots, and analysis												
	Presentation Preparation	Prepare slide deck, demo, and presentation rehearsal												
	Final Submission & Project Handover	Final submission and project handover (Deadline: Mar 31)												

Legend: Tasks are organized into 5 phases with detailed sub-tasks. The timeline spans 24 weeks from September 2025 to April 2026 with the final deadline on March 31, 2026.